



University of Idaho

Department of Computer Science

CS 4727/5727
Adversarial
Machine Learning

Dr. Alex Vakanski



Lecture 11

Privacy Attacks and Defenses



Lecture Content

11.1 Privacy Attacks in AML

11.1.1 Categories of Privacy Attacks

11.1.1.1 Membership Inference Attack

11.1.1.2 Feature Inference Attack

11.1.1.3 Model Extraction Attack

11.1.2 Causes of Privacy Leaks

11.1.3 Attacks against Distributed Learning

11.1.4 Privacy Attacks against LLMs

11.2 Defenses against Privacy Attacks

11.2.1 Anonymization Techniques

11.2.2 Encryption Techniques

11.2.3 Differential Privacy

11.2.4 Distributed Learning

11.2.5 ML-specific Techniques

Student presentation by Tom Poperszky

11.2.6 McKenna (2025) Scaling Laws for Differentially Private Language Models



Privacy Attacks in AML

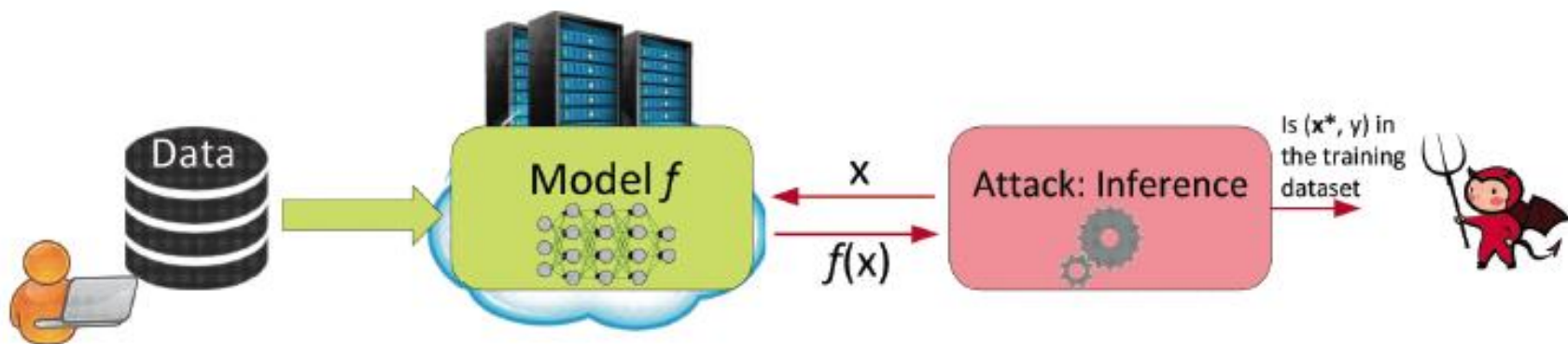
11.1 Privacy Attacks in AML

- *Privacy attacks* are also referred to as *inference attacks* or *confidentiality attacks*
- They can broadly be developed against:
 - Training data
 - E.g., reveal the identity of patients whose data was used for training a model
 - ML model
 - E.g., reveal the architecture and parameters of a model that is used by an insurance company for predicting insurance rates
 - E.g., reveal the model used by a financial institution for credit card approval
- Privacy attacks are commonly divided into the following *main categories*
 - Membership inference attack
 - Feature inference attack
 - Model extraction attack

Membership Inference Attack

11.1.1.1 Membership Inference Attack

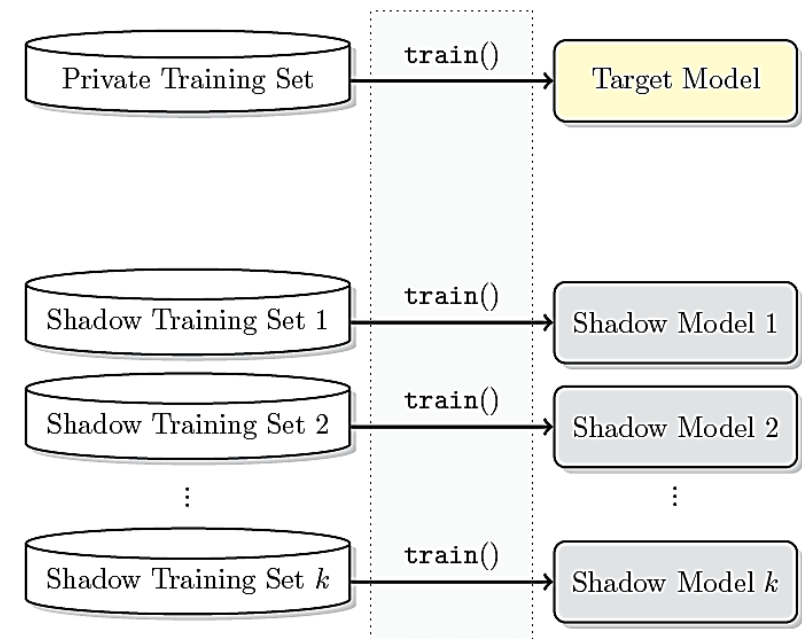
- *Membership inference attack*
 - Adversarial goal: determine whether or not an individual data instance x^* is part of the training dataset $\mathcal{D}$ for a model
- The attack typically assumes black-box query access to the model
- Attacks on both supervised classification models and generative models (GANs, VAEs) have been demonstrated
- A common approach is to first train several *shadow models* that imitate the behavior of the target model, and use the prediction vectors of the shadow models for training a binary classifier (that infers the membership)



Shadow Training Attack

11.1.1.1 Membership Inference Attack

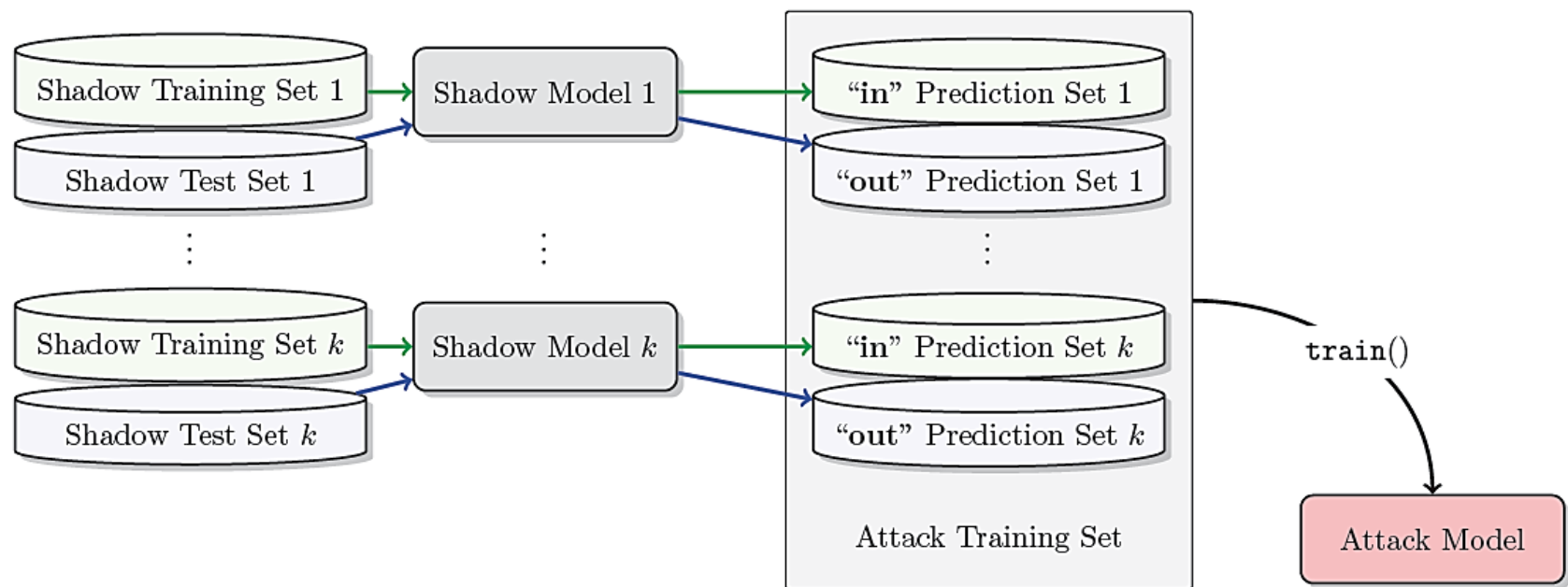
- [Shokri \(2016\) Membership Inference Attacks Against Machine Learning Models](#)
- Threat model:
 - The adversary has back-box query access to the target model
 - The goal is to infer whether input samples were part of its private training set
- *Shadow training* approach:
 - Create several **shadow models** to substitute the target model
 - Each shadow model is trained on a dataset that has a similar distribution as the private training dataset of the target model
 - E.g., if the target model performs celebrity face recognition as in our Assignment 2, the attacker can collect images of celebrities from the Internet
 - Then, query the target model with images of Jennifer Lopez, and if the confidence of the target model is high, then probably the private training set contains images of Jennifer Lopez: use those images for the shadow training sets
 - Same input instances are used to create shadow training sets for training multiple shadow models



Shadow Training Attack

11.1.1.1 Membership Inference Attack

- The output **probability vectors** from the shadow models are next used as inputs for training attack models (as binary classifiers) for each class
 - E.g., the probability vectors for all input images of Jennifer Lopez from all **shadow training sets** are labeled with 1 (meaning 'in' the training set)
 - The probability vectors for all input images of Jennifer Lopez from all **shadow test sets** are labeled with 0 (meaning 'out' or not in the training set)
 - An **attack model** is trained on these inputs to perform binary classification (in or out)
 - A separate attack model is trained for each celebrity person in the shadow training sets



Shadow Training Attack

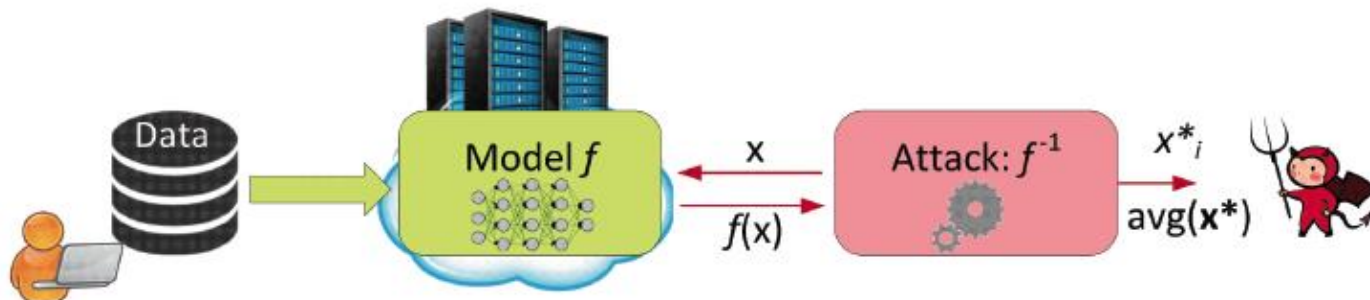
11.1.1.1 Membership Inference Attack

- The attack models for each class are afterward used to predict whether individual inputs instances were members of the private training set of the target model
- The assumption in this attack is that the output probability vectors of the shadow models are different for samples that are members of the shadow training sets, in comparison to samples that are members of the shadow test sets
- Experiments showed that increasing the number of shadow models improves the accuracy of membership inference, but it also increases the computational recourses

Feature Inference Attack

11.1.1.2 Feature Inference Attack

- *Feature inference attack*
 - Adversarial goal: recreate certain features of a data instance x^* or statistical properties (such as average value for a class) from the training dataset $\mathcal{D}$ for a model
- A.k.a. **attribute inference attack**, **reconstruction attack**, or **data extraction attack**
- Various attacks were developed to either recover partial information about the training data (such as sensitive features of the dataset, or typical representatives of specific classes in the dataset) or full data samples
 - An example of a training data extraction attack is described later in this lecture
- Similarly, recreating dataset properties that were not encoded in the dataset is also referred to as *property inference attack*
 - E.g., extract information about the ratio of men and women in a patient dataset, despite that gender information was not provided in the training records



Model Inversion Attack

11.1.1.2 Feature Inference Attack

- [Fredrickson \(2015\) Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures](#)
- *Model inversion attack* creates prototype examples for the classes in the dataset
 - The authors demonstrated an attack against a DNN model for face recognition
 - Given a person's name and white-box access to the model, the attack reverse-engineered the model and produced an averaged image of that person
 - The obtained averaged image (left image below) makes the person recognizable
 - This attack is limited to classification models where the classes pertain to one type of object (such as faces of the same person)

Recovered image
using the model
inversion attack

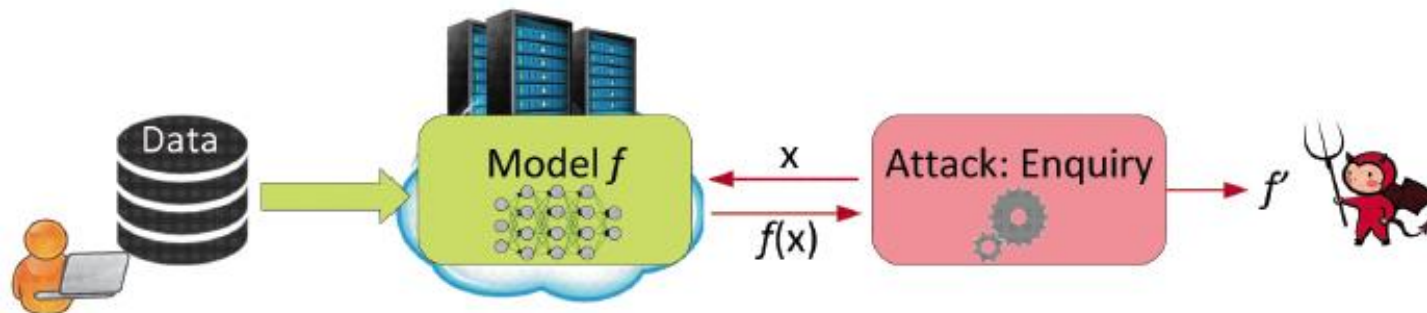


Image of the person
used for training the
model

Model Extraction Attack

11.1.1.3 Model Extraction Attack

- *Model extraction attack*
 - Adversarial goal: reconstruct an approximated model $f'(x)$ of the target model $f(x)$
- A.k.a. model inference attack
- The approximated function $f'(x)$ will act as a **substitute model** and produce similar predicted outputs as the target model
 - The adversary has black-box query access to the model
 - The goal is to “steal” the model and use the substitute model for launching other attacks, such as synthesis of adversarial examples, or membership inference attacks
- Besides creating a substitute model, several works focused on recovering the hyperparameters of the model, such as the number of layers, optimization algorithm, activation types used, etc.



Causes of Privacy Leaks

11.1.2 Causes of Privacy Leaks

- **Overfitting** is among the main causes of privacy leakage
 - It leads to **poor generalization** and memorization of the training data
 - Although adversarial training is often applied for increasing to model robustness, it reduces the accuracy on clean data, due to the trade-off between the model accuracy and robustness
 - The reduced accuracy can lead to increased sensitivity to data leakage
- **Datasets** that are more diverse and with larger number of class labels are more susceptible to attacks
 - I.e., binary classifiers are safer than multiclass models
 - Input samples that are **out-of-distribution** (i.e., are considered outliers with respect to the distribution of the training data) are more susceptible to privacy leakage
- **Model complexity** can also impact the vulnerability
 - Complex models with large number of parameters memorize more sensitive information about the training data

Attacks against Distributed Learning

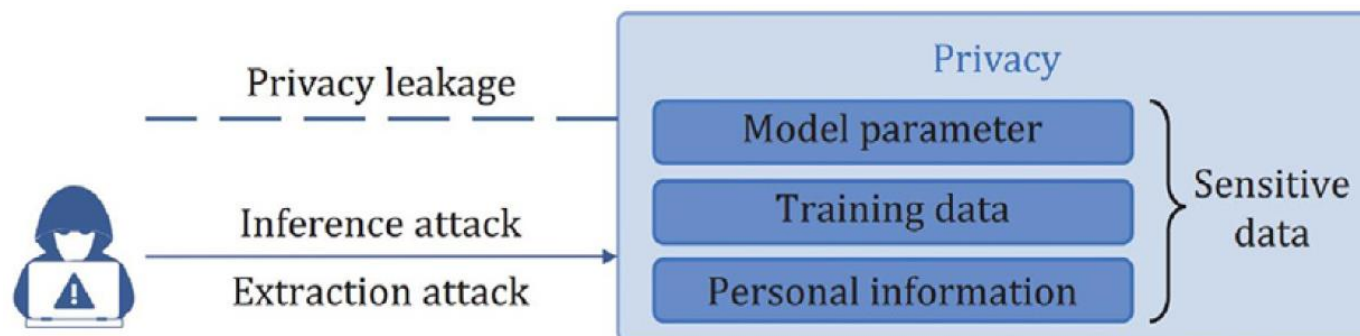
11.1.3 Attacks against Distributed Learning

- *Privacy attacks against federated learning* and related distributed learning models have also been demonstrated
- The attacks can be *passive* (the adversary collects the updates) and *active* (the adversary shares information to impact the training procedure)
 - A malicious attacker who participates in federated learning can perform **membership inference attack** to reveal if other participants used a data record for training
 - [Nasr \(2018\) Machine Learning with Membership Privacy Using Adversarial Regularization](#)
 - **Property inference attacks** were developed to reveal whether training data with certain properties were used by the other participants
 - [Melis \(2019\) Exploiting Unintended Feature Leakage in Collaborative Learning](#)
 - **Training data reconstruction attack** was accomplished by using an additional GAN model for reconstructing class representative samples from the local dataset used by the other participants
 - [Hitaj \(2017\) Deep Models Under the GAN: Information Leakage from Collaborative Deep Learning](#)

Privacy Attacks against LLMs

11.1.4 Privacy Attacks against LLMs

- LLMs face privacy issues that can undermine trust and reliability of their applications
 - *Privacy leakage* occurs when sensitive or personal information is inadvertently disclosed through the LLM's responses, due to the model's training on data containing such information
 - *Inference attacks* involve the exploitation of model responses to infer sensitive information about the training data or the underlying algorithms, potentially compromising LLM's privacy or security
 - *Extraction attacks* are attempts to reverse-engineer or obtain proprietary information, such as model parameters, by interacting with the LLM



Privacy Leakage

11.1.4 Privacy Attacks against LLMs

- *Privacy leakage* is an important concern due to the inherent potential of LLMs to inadvertently disclose sensitive information
 - For example, Samsung Electronics had sensitive corporate information unintentionally disclosed through interactions with ChatGPT
 - Also, Gemini Flash has been reported to collect excessive user data beyond what is necessary and has anonymization failures when sharing data with third parties
- Privacy leakage can arise from three primary causes
 - **Private information in training data**
 - The training datasets may contain sensitive or personally identifiable information where despite efforts at anonymization, there remains a risk of reidentification
 - **Data memorization**
 - LLMs can unintentionally retain and later reproduce specific pieces of training data, including sensitive personal information
 - **Inference leakage**
 - Even without explicit exposure to private data during interactions, LLMs can infer sensitive details through contextual clues
 - User prompts can trigger responses that reveal more than intended, especially when the model has learned correlations between different types of data during training

Inference Attack

11.1.4 Privacy Attacks against LLMs

- *Inference attacks* leverage the models' ability to infer sensitive information from their training datasets
 - **Membership inference attack (MIA)** – finds whether a particular data point was included in the model's training set
 - [Shi \(2023\) Detecting pretraining data from large language models](#)
 - Introduced a MIA detection technique called MIN-K% PROB
 - This method is based on the assumption that an unseen example is more likely to contain outlier words with low probabilities under LLM, whereas a seen example would exhibit fewer such low-probability terms
 - [Kaneko \(2024\) Sampling-based pseudo-likelihood for membership inference attacks](#)
 - SaMIA (Sampling-based pseudo-likelihood MIA) attack uses ROUGE-N score to quantify the n-gram match between the target text and multiple LLM outputs, thus inferring the text's membership in the training data

Inference Attack

11.1.4 Privacy Attacks against LLMs

- **Attribute inference attack**
 - Infer specific attributes or characteristics of the data contained within the training set of LLMs
 - E.g., reveal general information about the dataset, such as users demographic information
 - [Staab \(2023\) Beyond Memorization: Violating Privacy via Inference with Large Language Models](#)
 - Assessed the capabilities of nine LLMs to infer eight distinct personal attributes, including: location, income, age, sex, education, occupation, place of birth, relationship state
 - Constructed a dataset consisting of real Reddit profiles, and achieved up to 85% top-1 and 95% top-3 accuracy
 - The authors show that common mitigations, such as text anonymization and model alignment, are ineffective at protecting user privacy against LLM inference
- **Data reconstruction attack**
 - A more intrusive form of data compromise, aiming to reconstruct entire sequences or datasets from the model's knowledge base
 - [Carlini \(2020\) Extracting training data from large language models](#)
 - Explained later in this lecture



Model Extraction Attack

11.1.4 Privacy Attacks against LLMs

- *Model extraction attacks* are designed to retrieve information about the LLM, such the model parameters, gradients, architecture, etc.
 - This is typically achieved through an analysis of the model's outputs or observed behaviors
 - Attackers often employ methods such as optimization algorithms to fine-tune substitute models that emulate the behavior of the target model
 - [Carlini \(2024\) Stealing Part of a Production Language Model](#)
 - Explained later in the lecture

Training Data Extraction from GPT-2

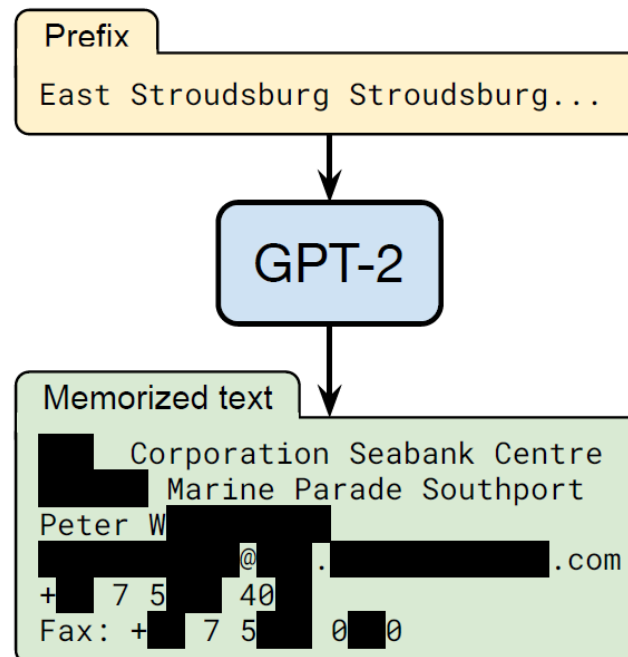
11.1.4 Privacy Attacks against LLMs

- *Training data extraction attack*
 - [Carlini \(2020\) Extracting training data from large language models](#)
- Attack on *GPT-2* language model (LM)
 - GPT-2 has 1.5 billion parameters, it is trained on public data collected from the Internet
- The goal of the attack is to analyze output text sequences from GPT-2 and identify text that has been memorized by the model
 - The authors had black-box query access to the GPT-2 model
- Successfully extracted data examples include:
 - Personally identifiable information (PII): names, phone numbers, e-mail addresses
 - News headlines, log files, Internet forum conversations, code
- The extracted information was present in just one document in the training data

Training Data Extraction from GPT-2

11.1.4 Privacy Attacks against LLMs

- Example of training data extraction
 - The authors query GPT-2 by entering the *prefix*: “East Stroudsburg Stroudsburg...”
 - The model outputted a block of text, which included the full name, phone number, email address, and physical address of the person
 - This information was included in the training data for GPT-2, it was memorized by the model, and extracted by using the training data extraction attack





Attack Threat Model

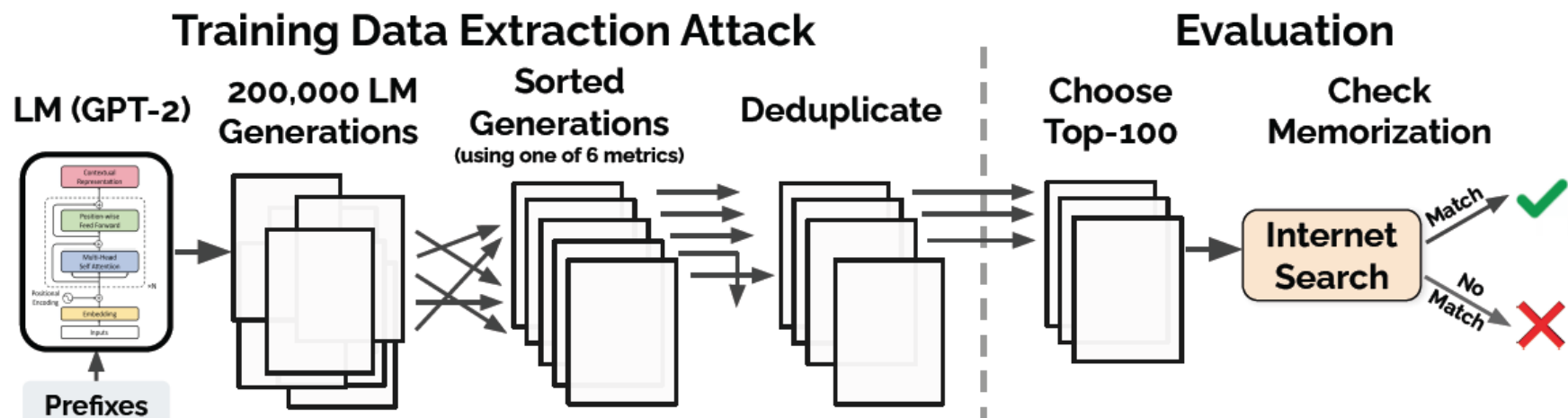
11.1.4 Privacy Attacks against LLMs

- The authors had black-box query access to GPT-2
- Objective: extract memorized training data
 - The strength of the attack is measured based on the number of documents in which the text appeared
 - Memorizing one word that occurred in many training examples (documents) is not severe
 - Stronger attack extract text that occurred in one single document
 - This is referred to as “**unintended**” memorization
- The training data for GPT-2 was collected by OpenAI from public sources
 - OpenAI didn't release the training dataset
 - But they released a document on the data collection process
 - The authors downloaded the public data by following the documentation
 - They didn't have access to the actual dataset used by OpenAI

Attack Approach

11.1.4 Privacy Attacks against LLMs

- Attack approach
 - Generate many text samples by using prefix prompts to GPT-2
 - Build 3 datasets of 200,000 generated samples, each sample is 256 tokens long
 - Sort the generated output text using 6 metrics (see next page)
 - LM has high confidence when the text is taken directly from the training data
 - Remove duplicate outputs
 - For the top 100 outputs, perform an Internet search to confirm whether the generated text is an exact match to a web document
 - Check with OpenAI to confirm if the extracted text occurred in their training dataset



Attack Approach

11.1.4 Privacy Attacks against LLMs

- *Metrics for sorting* the predicted text
 1. **Perplexity**, $P = \exp(- (1/n) \sum_{i=1}^n \log f_{\theta}(x_i | x_1, x_2, \dots, x_{i-1}))$
 2. Comparison to the predictions by GPT-Small and GPT-Medium models
 - It is less likely that the different models will memorize the same data
 3. Text entropy when the output is compressed using zlib compression
 4. Perplexity when the text is switched from uppercase to lowercase letters
 5. Averaged perplexity using a sliding window of 50 tokens
- Other strategies to improve the attack:
 - Use prompts based on Internet text
- The authors used 3 datasets of 200,000 generated samples
 - For the 6 metrics above, this resulted in 3×6 configurations, or 1,800 top samples

Results

11.1.4 Privacy Attacks against LLMs

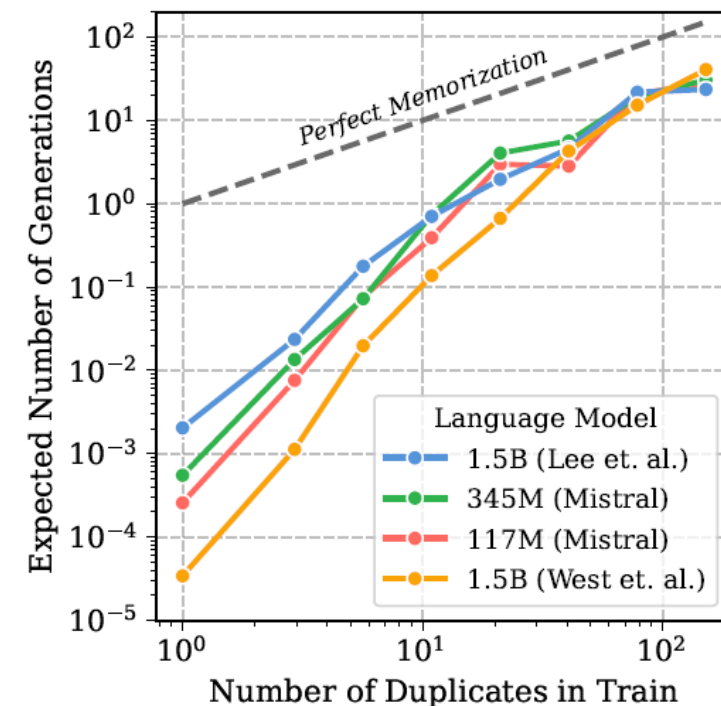
- The authors manually inspected 1,800 generated samples from GPT-2
- They identified 604 memorized training examples (about 33% of the samples)
 - The categories of memorized training examples are shown in the table

Category	Count
US and international news	109
Log files and error reports	79
License, terms of use, copyright notices	54
Lists of named items (games, countries, etc.)	54
Forum or Wiki entry	53
Valid URLs	50
Named individuals (non-news samples only)	46
Promotional content (products, subscriptions, etc.)	45
High entropy (UUIDs, base64 data)	35
Contact info (address, email, phone, twitter, etc.)	32
Code	31
Configuration files	30
Religious texts	25
Pseudonyms	15
Donald Trump tweets and quotes	12
Web forms (menu items, instructions, etc.)	11
Tech news	11
Lists of numbers (dates, sequences, etc.)	10

Data Deduplication as Mitigation Strategy

11.1.4 Privacy Attacks against LLMs

- *Defense against training data extraction using data deduplication*
 - [Kandpal \(2022\) Deduplicating Training Data Mitigates Privacy Risks in Language Models](#)
- This work showed the success of training data extraction attack against GPT-2 was due to the duplication of data in the training set
- Deduplicating data samples can significantly reduce the attack success
- The figure shows the expected number a text sequence can occur in generated text versus the number of occurrence of that text sequence in the training dataset
 - There is a linear relationship between these quantities
 - Perfect Memorization refers to generating a sequence at the same frequency as it appears in the training data





Stealing Part of LLM Attack

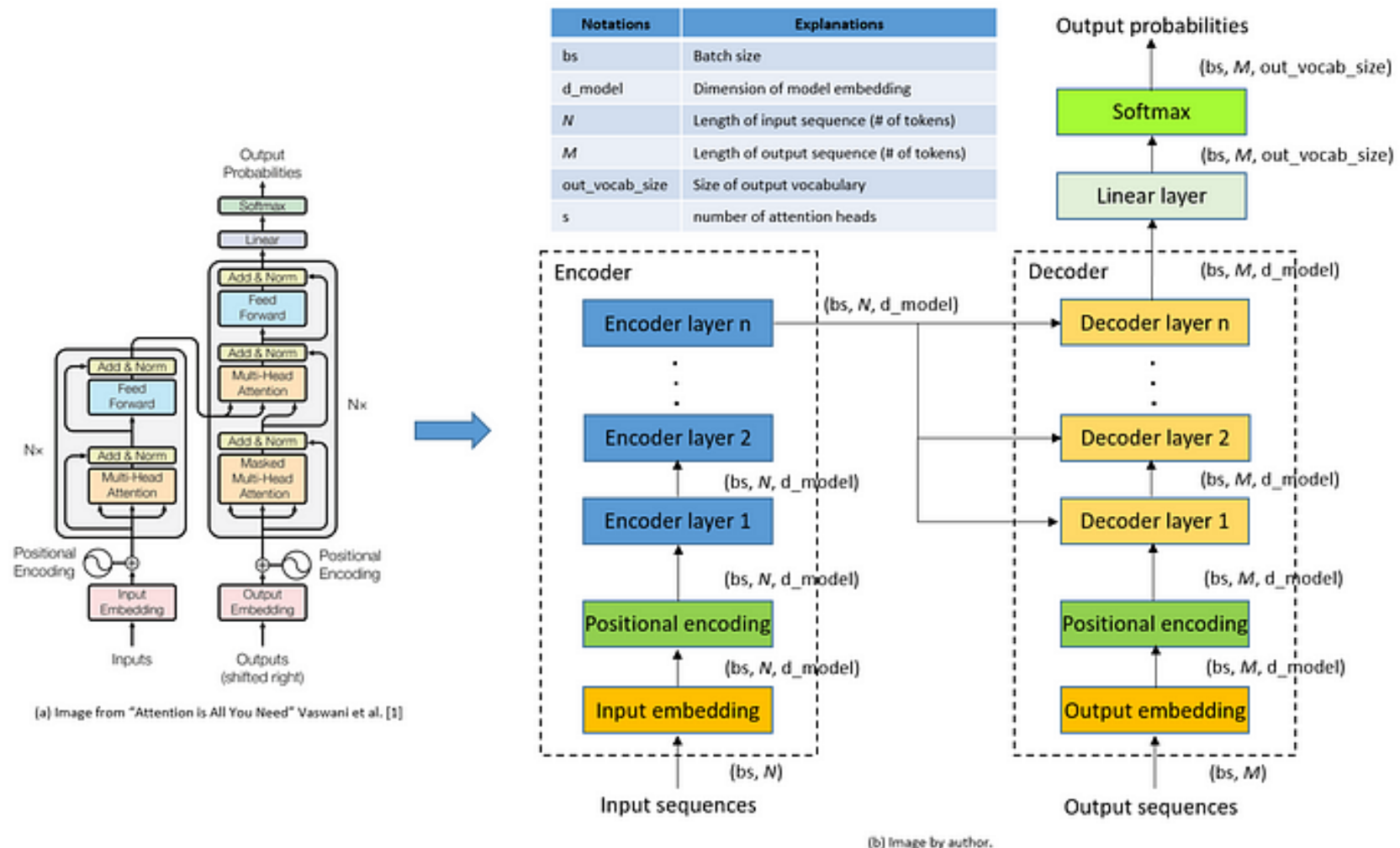
11.1.4 Privacy Attacks against LLMs

- *Model extraction attack*
 - [Carlini \(2024\) Stealing Part of a Production Language Model](#)
- Model-stealing attack extracts information from black-box production LLMs, such as Open AI's ChatGPT or Google's PaLM-2
 - The attack recovers the embedding projection layer (i.e., the last layer) of a transformer model
 - Black-box attack, the authors have typical API access to the LLMs
 - For under \$20 USD, the attack extracts the entire projection matrix of the OpenAI's GPT-3 LLMs ada and babbage

Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

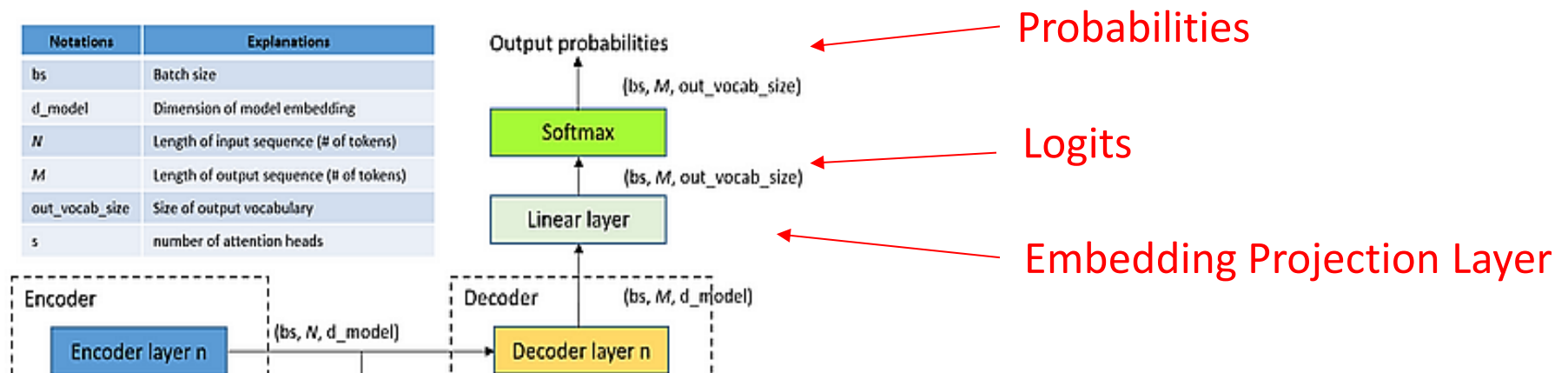
- Sizes of layers in transformer networks



Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

- Embedding projection layer
 - For each token, the layer takes a vector of size d_{model} and outputs a **logits** vector of size out_vocab_size
 - d_{model} is the size of vector embeddings (a.k.a. hidden dimension), i.e., it is the dimensionality of all internal vectors processed throughout the model (see previous page)
 - out_vocab_size is the size of the vocabulary (typical LLM has about 50,000 words vocabulary)
 - During training the model predicts the next token at every position
 - Logits of tokens at position t are used to predict the token at position $t+1$
 - E.g., the logits for the tokens “Mary had a little” are used to predict the logit vector for the next token
 - The logit vector has 50,000 values, with the highest value having the word “lamb”



Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

- Attack assumption: querying the LLM via API returns the logits values for each token (having the size `out_vocab_size`)
 - The embedding dimension `d_model` is not known for proprietary LLMs
 - Note that `d_model` is much smaller than the vocabulary size
- Recovering the size of the embedding dimension
 - Intuition: although the logits have `out_vocab_size`, they lie in a smaller dimensional space with size `d_model`
 - By querying the LLM many times with random prompts and applying Singular Value Decomposition, the dimensionality of the reduced space can be estimated
 - Algorithm:

Algorithm 1 Hidden-Dimension Extraction Attack

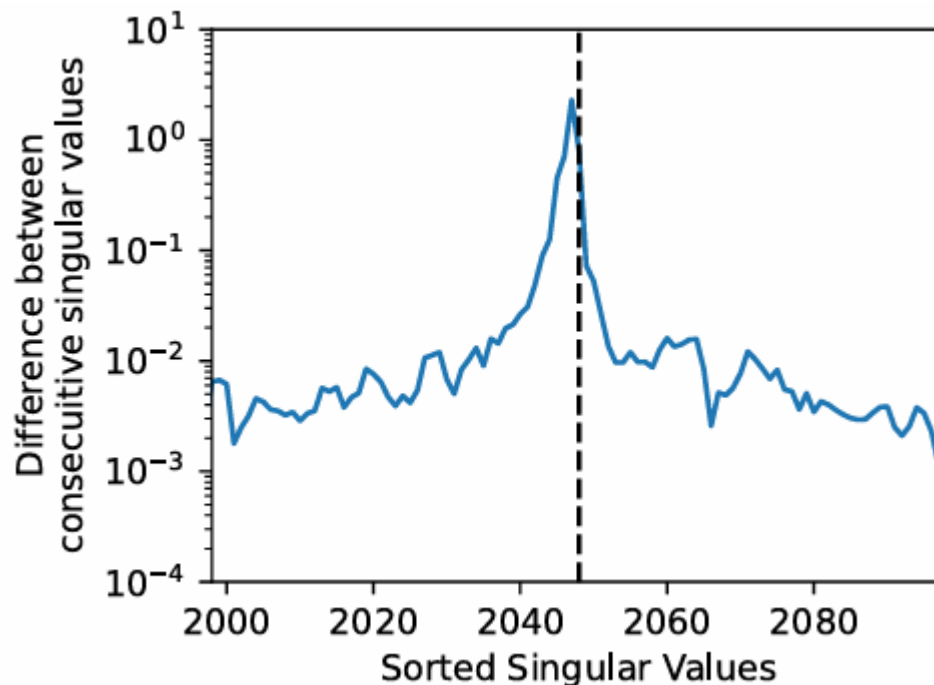
Require: Oracle LLM $\mathcal{O}$ returning **logits**

- 1: Initialize n to an appropriate value greater than h
 - 2: Initialize an empty matrix $\mathbf{Q} = \mathbf{0}^{n \times l}$
 - 3: **for** $i = 1$ to n **do**
 - 4: $p_i \leftarrow \text{RandPrefix}()$ ▷ Choose a random prompt
 - 5: $\mathbf{Q}_i \leftarrow \mathcal{O}(p_i)$
 - 6: **end for**
 - 7: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \leftarrow \text{SingularValues}(\mathbf{Q})$
 - 8: $\text{count} \leftarrow \arg \max_i \log \|\lambda_i\| - \log \|\lambda_{i+1}\|$
 - 9: **return** count
-

Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

- Recovering the size of the embedding dimension
 - The figure shows the sorted singular values for the Pythia 1.4B model
 - The largest SV is at 2,048, which is the embedding dimension for this model
 - The table shows the recovered embedding dimensions for several LLMs



Model	Hidden Dim
GPT-2 Small (fp32)	768
GPT-2 XL (fp32)	1600
Pythia-1.4 (fp16)	2048
Pythia-6.9 (fp16)	4096
LLaMA 7B (fp16)	4096
LLaMA 65B (fp16)	8192

Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

- Recovering the weights of the embedding projection layer
 - The authors used the SVD decomposition to further recover the entire projection matrix of the last layer
- The table below shows the attack success rate and the cost for querying LLMs from OpenAI for dimension extraction and weight matrix extraction

Model	Dimension Extraction			Weight Matrix Extraction		
	Size	# Queries	Cost (USD)	RMS	# Queries	Cost (USD)
OpenAI ada	1024 ✓	$< 2 \cdot 10^6$	\$1	$5 \cdot 10^{-4}$	$< 2 \cdot 10^7$	\$4
OpenAI babbage	2048 ✓	$< 4 \cdot 10^6$	\$2	$7 \cdot 10^{-4}$	$< 4 \cdot 10^7$	\$12
OpenAI babbage-002	1536 ✓	$< 4 \cdot 10^6$	\$2	†	$< 4 \cdot 10^6$ ††	\$12
OpenAI gpt-3.5-turbo-instruct	* ✓	$< 4 \cdot 10^7$	\$200	†	$< 4 \cdot 10^8$ ††	\$2,000 ††
OpenAI gpt-3.5-turbo-1106	* ✓	$< 4 \cdot 10^7$	\$800	†	$< 4 \cdot 10^8$ ††	\$8,000 ††

✓ Extracted attack size was exactly correct; confirmed in discussion with OpenAI.

* As part of our responsible disclosure, OpenAI has asked that we do not publish this number.

† Attack not implemented to preserve security of the weights.

†† Estimated cost of attack given the size of the model and estimated scaling ratio.



Stealing Part of LLM Attack

11.1.4 Privacy Attacks against LLMs

- Usefulness of this attack
 - It reveals the width of the LLM, which is often correlated with the number of total parameters
 - It reduces the degree of black-box knowledge about the model
 - Raises concerns that advanced attacks may recover more information about the model
- Responsible disclosure
 - The authors shared the attack with developers of LLMs
 - In response, OpenAI and Google modified their APIs to introduce mitigations and defenses against the attack

Mitigation Strategies for LLMs

11.1.4 Privacy Attacks against LLMs

- Mitigation strategies for privacy attacks on LLMs include
 - **Selecting the training data** for LLMs
 - Apply methods that limit the amount of sensitive content (e.g., filter personal information)
 - De-duplicate content in the training data
 - **Training with differential privacy**
 - DP can reduce, but cannot prevent, memorization of content in the dataset
 - **Federated learning**
 - Enables models to be trained on decentralized data, reducing the need to centralize and expose sensitive user data
 - **Cryptography methods**
 - Encryption techniques, secure communication protocols to prevent exposing model parameters or user's inputs
 - **Watermarking**
 - Embeds unique identifiers into data, serving as a traceable source against unauthorized use or access (e.g., by integrating special tokens for sensitive information)
 - **Auditing LLMs**
 - Apply audits to determine the level of memorization in LLMs
- Limitations: reduced accuracy, longer training times

Defenses against Privacy Attacks

11.2 Defenses against Privacy Attacks

- **Data privacy** techniques have the goal of allowing analysts to learn about *trends* in data, without revealing information specific to *individual data instances*
 - Therefore, privacy techniques involve an **intentional** release of information, and attempt to control what can be learned from the released information
- Related to data privacy is the **Fundamental Law of Information Recovery**, which states that “*overly accurate estimates of too many statistics can completely destroy privacy*”
 - I.e., extracting useful information from a dataset (e.g., for training an ML model) poses a privacy risk to the data
- There is an inevitable trade-off between privacy and accuracy (i.e., utility)
 - Preferred privacy techniques should provide an estimate of how much privacy is lost by interacting with data



Defenses against Privacy Attacks

11.2 Defenses against Privacy Attacks

- Defense strategies against privacy attacks in ML can be broadly classified into:
 - Anonymization techniques
 - Encryption techniques
 - Differential privacy
 - Distributed learning
 - ML-specific techniques

Anonymization Techniques

11.2.1 Anonymization Techniques

- *Anonymization* techniques provide privacy protection by removing identifying information in the data
- E.g., remove personal identifiable information (PII)
 - In the example below, the Name and Address columns are removed

User ID	Name	Address	Account Type	Subscription Date
001	Alice	123 A St	Pro	01/02/20
002	Bob	234 B St	Free	02/03/21
003	Charlie	456 C St	Pro	03/04/18

Anonymization Techniques

11.2.1 Anonymization Techniques

- Anonymization is not an efficient defense method, since the remaining information in the data can be used for identifying the individual data instances
 - For example, based on health records (including diagnoses and prescriptions) with removed personal information released by an insurance group in 1997, a researcher extracted the information for the Governor of Massachusetts
 - This is referred to as *de-anonymization*
 - The same researcher later showed that 87% of all Americans can be uniquely identified using 3 bits of information: ZIP code, birth date, and gender

Dataset 1: Users medical database

User ID	Name	Address	Zip Code	Birth date	Gender	Probable disease ID
001	Alice	123 A St	83401	01/02/1997	F	120
002	Bob	234 B St	83402	02/03/1995	M	35
003	Charlie	456 C St	83403	03/04/1999	M	240

Dataset 2: Users medical database with name and address removed

User ID	Zip Code	Birth date	Gender	Probable disease ID
001	83401	01/02/1997	F	120
002	83402	02/03/1995	M	35
003	83403	03/04/1999	M	240



Linkage Attack

11.2.1 Anonymization Techniques

- De-anonymization of data by using connections to external sources of information is referred to as *linkage attack*
 - For example:
 - In 2006, Netflix published anonymized 10 million movie rankings by 500,000 customers
 - Two researchers showed later that by using movie recommendations on IMDb (Internet Movie Database) they could identify the customers in the Netflix data

Dataset 1: Anonymized dataset with removed personal information

User ID	Name	Address	Account Type	Subscription Date
001			Pro	01/15/20
002			Pro	02/03/21
003			Free	03/04/18

Dataset 2: External public dataset that reveals the users in Dataset 1

User ID	Product Name	Product Price	Purchase Date
001	TV	400	01/02/20
002	Iphone	1,199	02/02/21
003	Watch	130	02/22/18

k -anonymity

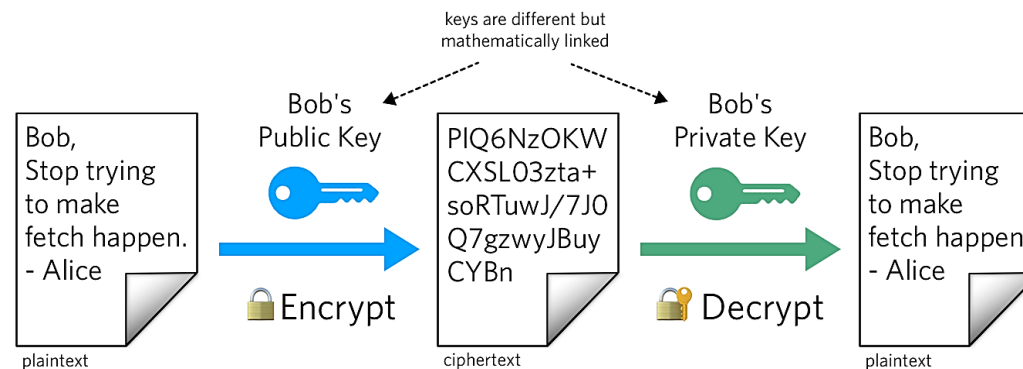
11.2.1 Anonymization Techniques

- k -anonymity is an approach for protecting data privacy by suppressing certain identifying data features
 - This approach removes fields of data for individuals who have unique characteristics
 - E.g., students at UI who are from Latvia and are enrolled in Architecture
- A dataset is k -anonymous if for any person's record, there are at least $k - 1$ other records that are indistinguishable
- Limitation: this approach is mostly applicable to large datasets with low-dimensional input features
 - The more input features there are for each record, the higher the possibility of unique records

Encryption Techniques

11.2.2 Encryption Techniques

- **Encryption** is a cryptography approach, which converts the original representation of information into an alternative form
 - The sender of encrypted information shares the decoding technique only with the intended recipients of the information
- **Encrypting the training data** has been applied in ML
 - Common techniques for data encryption include:
 - Homomorphic encryption (HE)
 - Secure multi-party computation (SMPC)
- **Encrypting ML models** is less common approach
 - Homomorphic encryption has been applied to the model gradients in collaborative DL setting to protect the model privacy





Homomorphic Encryption

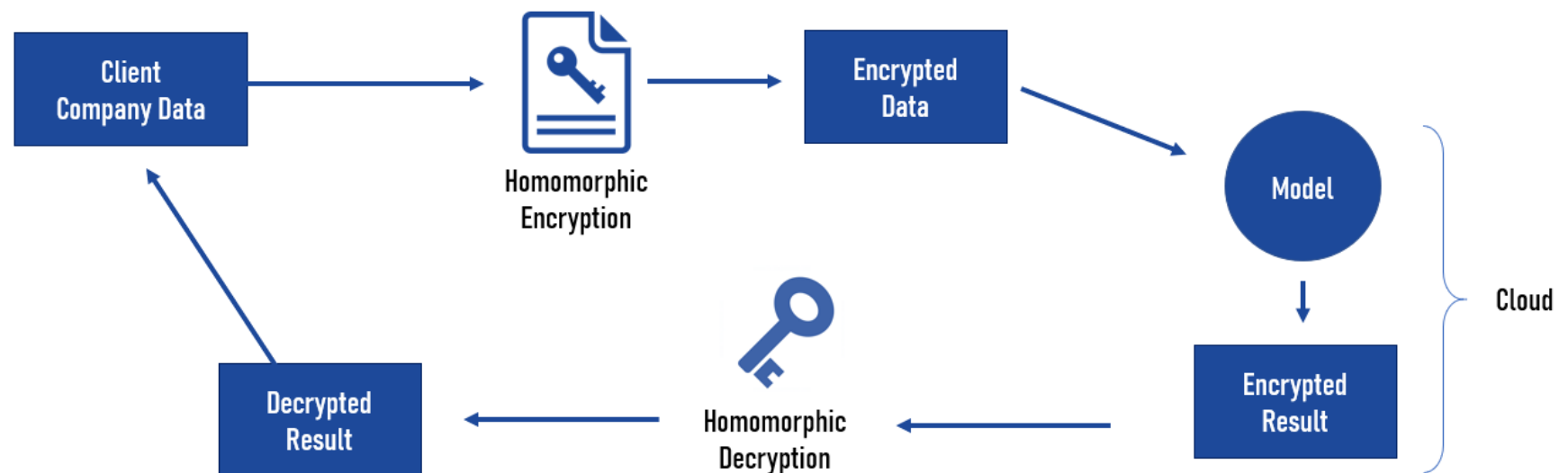
11.2.2 Encryption Techniques

- *Homomorphic encryption (HE)* allows users to perform **computations on encrypted data** (without decrypting it)
 - Encrypted data can be analyzed and manipulated without revealing the original data
- HE uses a public key to encrypt the data, and applies an algebraic system (e.g., additions and multiplications) to allow computations while the data is still encrypted
 - Only the person who has a matching **private key** can access the decrypted results

Homomorphic Encryption

11.2.2 Encryption Techniques

- In ML, training data can be encrypted and send to a server for model training
 - Even if the server is untrusted or it is compromised, confidentiality of the data will remain preserved
 - One main limitation of HE is the slowing down of the training process
- HE has been applied to traditional ML approaches, such as Naïve Bayes, Decision Trees
 - Training DNNs over encrypted data is still challenging, due to the increased computational complexity



Privacy versus Confidentiality

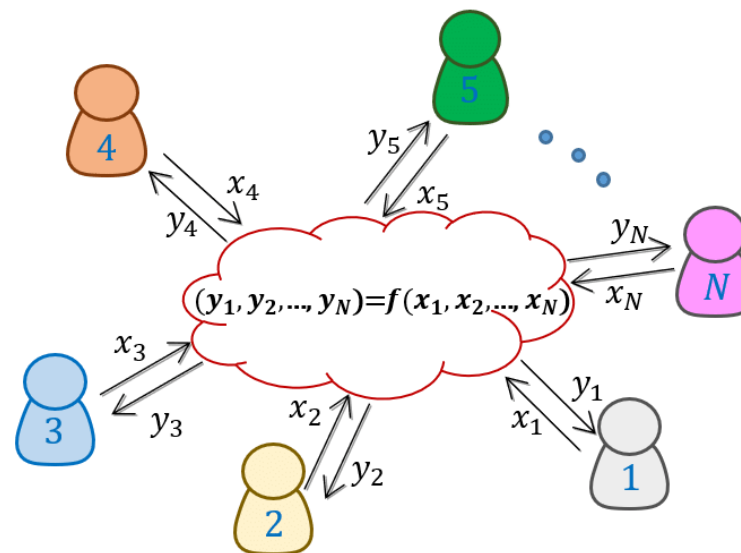
11.2.2 Encryption Techniques

- Encryption techniques in ML are mainly applied to protect the confidentiality of the data or model
- **Confidentiality** refers to keeping the information (training data, model parameters) hidden from the clients and the public
 - It is ensuring that only authorized parties have access to the information
 - E.g., a server has an ML model trained on private data and provides the model to a client for inference
 - It is preferred to preserve the confidentiality of the model parameters from the client
- **Privacy** refers to intentional release of information in a controlled manner to prevent unintended information leakage
 - It is ensuring that released data cannot uniquely identify individual inputs
 - E.g., a server applies Differential Privacy to a trained ML model to prevent information leakage about individual inputs
- Protecting privacy is more challenging than protecting confidentiality

Secure Multi-Party Computation

11.2.2 Encryption Techniques

- **Secure Multi-Party Computation** (SMPC) is an extension of encryption in multi-party setting
 - SMPC allows two or more parties to jointly perform computation over their private data, without sharing the data
 - E.g., two banks want to know if they have both flagged the same individuals and learn about the activities by those individuals
 - The banks can share encrypted tables of flagged individuals, and they can decrypt only the matched records, but not the information for individuals that are not in both tables



Secure Multi-Party Computation

11.2.2 Encryption Techniques

- SMPC versus HE
 - **SMPC** protects the **privacy** of the data in collaborative learning
 - E.g., participants in collaborative learning do not trust the other participants or the central server
 - **HE** protects the **confidentiality** of the data from external adversaries
 - E.g., a data owner wants to use a MLaaS (Machine Learning as a Service), but does not trust the service provider: (1) the owner sends encrypted data, (2) the provider processes encrypted data and sends back encrypted results, (3) the owner decrypts the results
 - Or, a bank can store encrypted banking information in the cloud, and use HE to ensure that only the employees of the bank can access the data



Secure Multi-Party Computation

11.2.2 Encryption Techniques

- In ML, SMPC can be used to compute updates of the model parameters by multiple parties that have access to their private data
 - For examples, SMPC has been applied to federated learning, where participants encrypt their updates, and the central server can recover only the sum of the updates from all participants
 - Beside the data privacy, SMPC also offers protection against adversarial participants
 - Either all parties are honest and can jointly compute the correct output, or if a malicious party is dishonest the joint output will be incorrect
- SMPC has been applied to traditional ML models, such as decision trees, linear regression, logistic regression, Naïve Bayes, k -means clustering
 - Application of SMPC to deep NNs is challenging, due to increased computational costs



Differential Privacy

11.2.3 Differential Privacy

- *Differential privacy* is based on employing obfuscation mechanisms for privacy protection
 - A **randomization mechanism** $\mathcal{M}(D)$ applies noise ξ to the outputs of a function $f(D)$ to protect the privacy of individual data instances, i.e., $\mathcal{M}(D) = f(D) + \xi$
 - Commonly used randomization mechanisms include Laplacian, Gaussian, and Exponential mechanism
- DP is often implemented in practical applications
- Examples include:
 - Google, deployed in Chrome for collecting usage statistics
 - Apple, improving its Intelligent Personal Assistant Siri suggestions
 - Microsoft, privacy-preserving telemetry collection in Windows
 - LinkedIn, for advertiser analytics and reposting
 - Uber, released open-sourced DP tools for secure data analysis
 - U.S. Census Bureau, for demographic data release

DP Example

11.2.3 Differential Privacy

- Consider two databases D_1 and D_2 that show if a person has diabetes or not
 - The only difference between the two databases is that D_2 does not include the last record in D_1 (for Bob)
- Let's assume that the databases are publicly available for making queries
 - To protect patient identities, it is not allowed to query the patient names
- However, an adversary can query the sum of the persons with diabetes in the first database (e.g., $f(D_1) = 64$), and the sum in the second database (e.g., $f(D_2) = 63$)
 - Based on the difference $f(D_1) - f(D_2) = 64 - 63 = 1$, the adversary can infer that Bob has diabetes
 - Alternatively, if $f(D_1) = 63$ and $f(D_2) = 63$, the adversary can infer that Bob does not have diabetes

D_1 (includes Bob)

Name	Has Diabetes
Don	1
Monica	0
...	
...	
Chris	1
Bob	1

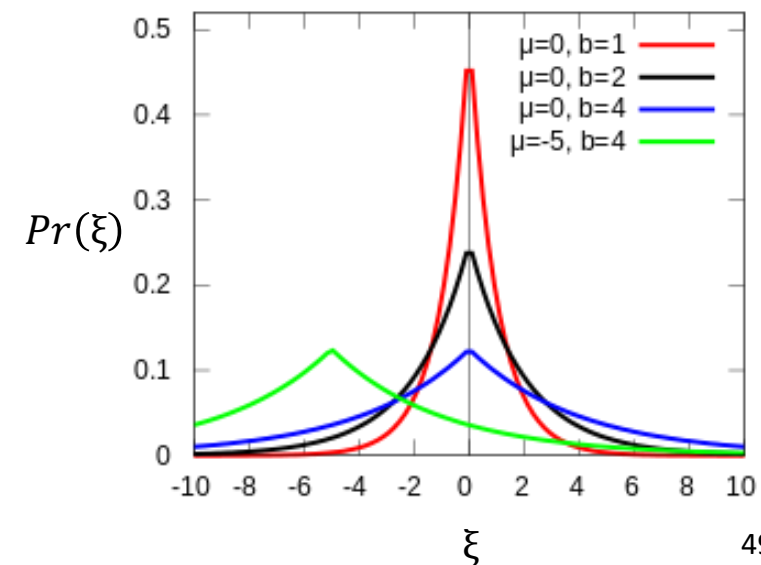
D_2 (without Bob)

Name	Has Diabetes
Don	1
Monica	0
...	
...	
Chris	1

DP Example (cont'd)

11.2.3 Differential Privacy

- An algorithm that is *differentially private* adds noise to the answers for $f(D_1)$ and $f(D_2)$ to make it difficult to infer the information about Bob
 - I.e., a **randomization mechanism** $\mathcal{M}(D)$ is selected to add noise ξ to the output answers to queries $f(D)$, that is, $\mathcal{M}(D) = f(D) + \xi$
- Additive noise ξ from a **Laplacian distribution** (shown) is commonly applied
 - E.g., let's assume a **privacy budget** $\epsilon = 0.5$ and let's sample noise from a Laplacian distribution with $\mu = 0$ and scale $b = 1/\epsilon = 1/0.5 = 2$
 - 6 random noise samples are: $\xi \in \{-0.13, 2.06, -1.67, -2.49, -0.52, 0.37\}$
 - Consider 3 queries by the adversary having the outputs $f(D_1) = 64$ and $f(D_2) = 63$ with added Laplacian noise ξ :
 - $\mathcal{M}(D_1) - \mathcal{M}(D_2) = 63.87 - 65.06 = -1.19$
 - $\mathcal{M}(D_1) - \mathcal{M}(D_2) = 62.33 - 60.51 = 1.82$
 - $\mathcal{M}(D_1) - \mathcal{M}(D_2) = 63.48 - 63.37 = 0.11$
 - Based on the differences between the randomized outputs from the queries for D_1 and D_2 , now it is impossible for the adversary to tell if Bob has diabetes



DP Mechanism

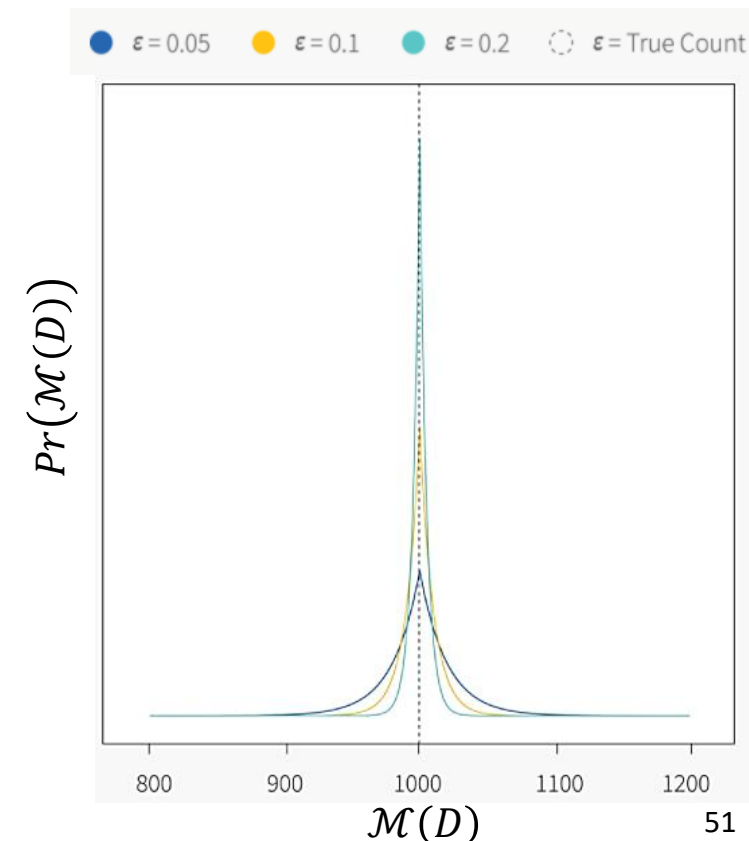
11.2.3 Differential Privacy

- The important question in DP is: **how much noise to add?**
 - The amount of noise ξ depends on the data, and it needs to be adjusted
 - E.g., a function $f_1(D)$ that provides the yearly income of people in thousands of dollars would require different level of noise than a function $f_2(D)$ that provides the height in feet
- The **sensitivity** of the function f determines how much the output $f(D)$ changes by adding a single data instance
 - Sensitivity is defined as $\Delta f = \max \|f(D_1) - f(D_2)\|_1$ for all possible datasets D_1 and D_2 differing in one data instance, where $\|\cdot\|_1$ denotes ℓ_1 -norm
 - E.g., for the example with medical diabetes records, the sensitivity is $\Delta f = 1$, since the sum of the people with diabetes can change only by 1 when a single input is added
- A Laplacian mechanism that is **ϵ -differentially private** adds a Laplacian noise with scale $b = \Delta f / \epsilon$
- Note that **if the privacy budget ϵ has smaller values**, this will result in larger amount of Laplacian noise ξ added to $f(D)$
 - Thus, the noisy outputs $\mathcal{M}(D)$ will reveal less private information about the inputs (i.e., provide better privacy protection), but also the noisy answers to the queries $\mathcal{M}(D)$ will be less accurate

DP with Laplacian Randomization

11.2.3 Differential Privacy

- The figure shows the **probability distributions** of the outputs $\mathcal{M}(D)$ for three different levels of Laplacian noise with $\epsilon \in \{0.05, 0.1, 0.2\}$
 - The true output value is $f(D) = 1,000$
 - Larger values of ϵ have distributions that are tighter around the true value of $f(D) = 1,000$ in the figure, and hence are more accurate, but leak more privacy
- A mechanism $\mathcal{M}(D)$ is **ϵ -differentially private** if for all databases D_1 and D_2 that differ by at most one instance, and for any subset of outputs S :
$$\Pr(\mathcal{M}(D_1) \in S) \leq e^\epsilon \Pr(\mathcal{M}(D_2) \in S)$$
 - In other words, ϵ -differential privacy ensures that the probabilities of any two outputs $\mathcal{M}(D_1)$ and $\mathcal{M}(D_2)$ differ by at most e^ϵ
 - E.g., for $\epsilon = 0.05$, $\Pr(\mathcal{M}(D_1))/\Pr(\mathcal{M}(D_2))$ is at most $e^{0.05} = 1.05$
 - Smaller ϵ ensures less similar outputs $\mathcal{M}(D_1)$ and $\mathcal{M}(D_2)$, and provides higher levels of privacy



DP with Gaussian Randomization

Differentially Private SGD

- There are other DP mechanisms besides the Laplacian mechanism, that are more suitable for some applications
- The *Gaussian mechanism* adds Gaussian noise instead of Laplacian noise, and the level of noise is based on the ℓ_2 -norm sensitivity, instead of ℓ_1 -norm
- A Gaussian mechanism is (ϵ, δ) -differentially private if for all databases D_1 and D_2 that differ by at most one instance, and for any subset of outputs S :

$$Pr(\mathcal{M}(D_1) \in S) \leq e^\epsilon Pr(\mathcal{M}(D_2) \in S) + \delta$$

- The (ϵ, δ) -differential privacy that is provided by the Gaussian mechanism introduces the *probability parameter δ*
 - Informally, (ϵ, δ) -differential privacy is guaranteed with probability $1 - \delta$
 - E.g., for $\delta = 0.05$, the method is ϵ -differentially private with 95% probability
- The Gaussian mechanism is therefore weaker than the Laplacian mechanism, since it allows scenarios when the privacy cannot be guaranteed



DP in Machine Learning

11.2.3 Differential Privacy

- Training ML models can be considered an extension of the previous example on querying databases
 - I.e., ML models use data to learn a function, which is afterward used for prediction
- The datasets for training ML models often contain sensitive information (e.g., medical records, personal information), so it is important to provide privacy guarantees
 - On the other hand, we know that ML models can memorize the training data, which can be exploited by adversaries to recover information about the data from a trained model
- The challenge is: how to extract enough information from data to train accurate ML models without revealing the data

DP in Machine Learning

11.2.3 Differential Privacy

- In ML, DP is achieved by adding noise to:
 - *Model parameters*
 - Several works applied DP to conventional ML methods
 - **Differentially private SGD** (Abadi, 2016) clips and adds noise to the gradients of deep NNs during training
 - This reduces the memorization of individual input instances by the model
 - The approaches that apply obfuscation to the model parameters via DP are also referred to as **differentially private ML**
 - *Model outputs*
 - **PATE** (Private Aggregation of Teacher Ensembles) approach (Papernot, 2018) employs an ensemble of models trained on disjoint subsets of the training data, called teacher models
 - Noise is added to the outputs of the teacher models, and the aggregated outputs are used to train another model, called student model
 - *Training data*
 - Obfuscation of training data in ML has been also investigated in several works

DP in Machine Learning

11.2.3 Differential Privacy

- DP is typically applied in a *centralized learning setting*, where the data and model are at the same location
 - In this scenario, all data is gathered in one central location for model training
 - E.g., MLaaS typically requires that the users upload their data to a cloud-based server for training a model
- Recently, DP has also been applied in a *distributed learning setting*, where the data are kept at separate locations from the model
 - **DP-FedAvg** (McMahan, 2018) is applied to federated learning
 - It introduced the Federated Averaging algorithm to limit the contributions by the data from individual users to the learning model

Differentially Private SGD

11.2.3.1 Differentially Private SGD

- [Abadi \(2016\) Deep Learning with Differential Privacy](#)
- This work introduced differential privacy (DP) for training ML models for protecting the privacy of the training data
 - *Differential privacy (DP)* is applied to *Stochastic Gradient Descent (SGD)* during model training
 - DP-SGD clips the gradients and adds Gaussian noise to the gradients with respect to the model parameters
 - This approach controls the amount of information from the training data that is memorized by the model during training
 - The goal is to produce ML models which provide approximately the same privacy when an individual input instance is removed from the training dataset
- The paper also introduces a method for calculating the privacy loss, called *moments accountant*

Differentially Private SGD

11.2.3.1 Differentially Private SGD

- Differentially Private Stochastic Gradient Descent (DP-SGD)
 - DP-SGD adds two additional steps to SGD: Clip gradient, and Add noise

Algorithm 1 Differentially private SGD (Outline)

Input: Examples $\{x_1, \dots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N} \sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t , noise scale σ , group size L , gradient norm bound C .

Initialize θ_0 randomly

for $t \in [T]$ **do**

Take a random sample L_t with sampling probability L/N

Compute gradient

For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t} \mathcal{L}(\theta_t, x_i)$

Clip gradient

$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C})$

Add noise

$\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L} (\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}))$

Descent

$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$

Output θ_T and compute the overall privacy cost (ϵ, δ) using a privacy accounting method.

Introduced DP steps

Gradient Clipping

11.2.3.1 Differentially Private SGD

- ML models tend to memorize more information about some input samples than others
 - Input samples that produce large gradients are prone to be memorized by the model, and violate privacy
- DP-SGD approach proposes to clip the ℓ_2 norm of the gradient to a **threshold C** , in order to limit the influence by the individual input samples
 - Also, since the values of the gradients cannot be estimated ahead of time, the clipping operation controls the sensitivity of the DP randomization mechanism
- If the gradient at step t by an input sample x_i is $\mathbf{g}_t(x_i) = \nabla_{\theta_t} \mathcal{L}(\theta_t, x_i)$, the clipped gradient $\bar{\mathbf{g}}_t(x_i)$ is:

$$\bar{\mathbf{g}}_t(x_i) = \begin{cases} \mathbf{g}_t(x_i) & \text{if } \|\mathbf{g}_t(x_i)\|_2 \leq C \\ \frac{\mathbf{g}_t(x_i)}{\|\mathbf{g}_t(x_i)\|_2 / C} & \text{if } \|\mathbf{g}_t(x_i)\|_2 > C \end{cases}$$

- That is, if the norm $\|\mathbf{g}_t(x_i)\|_2$ is greater than C , the gradient is scaled down to have a norm equal to C

Adding Noise

11.2.3.1 Differentially Private SGD

- GP-SGD approach employs a Gaussian randomization mechanism
- Gaussian noise is added to the gradients at each training step t , according to:

$$\tilde{\mathbf{g}}_t = \frac{1}{L} \left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}) \right)$$

- At each step t , the average of the clipped gradient for a batch of inputs (with a batch size L) is first calculated as $\frac{1}{L} \sum_i \bar{\mathbf{g}}_t(x_i)$
- Gaussian noise $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ with mean 0 and diagonal co-variance $\sigma^2 C^2$ is afterward added to the batch-averaged gradient
 - Note that the co-variance is a function of the clipping threshold C
 - E.g., larger value of C does less clipping, but requires more noise to achieve the same level of privacy

Moments Accountant

11.2.3.1 Differentially Private SGD

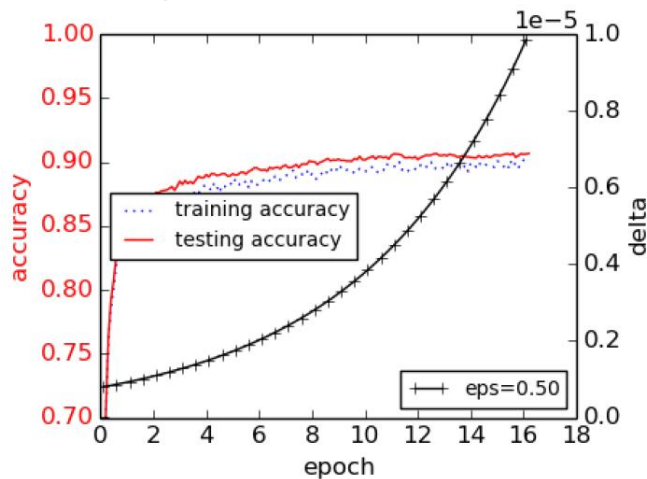
- The **composition property in DP** states that if the privacy loss (budget) for one interaction with the data is ϵ_1 and for another interaction with the data is ϵ_2 , the combined privacy loss is $\epsilon_1 + \epsilon_2$
 - Therefore, by accumulating the privacy loss for each mini-batch when training an ML model, it is possible to calculate the overall privacy loss during training
- **Moments accountant** is an introduced approach in the paper that evaluates the **privacy loss** of a model training with DP-SGD
 - The privacy loss is estimated at each training step, and it is used to calculate the cumulative privacy loss over all training epochs
 - Note that increasing the number of training epochs increases the privacy loss
 - E.g., training a model for 100 epochs that achieved a privacy loss of $\epsilon = 1.26$, when training for 400 epochs the privacy loss increased to $\epsilon = 2.55$
- Moments accountant employs the moments of mixtures of Gaussian distributions to calculate the upper bound of the cumulative privacy loss
 - The approach is described in more detail in the paper

Experimental Evaluation

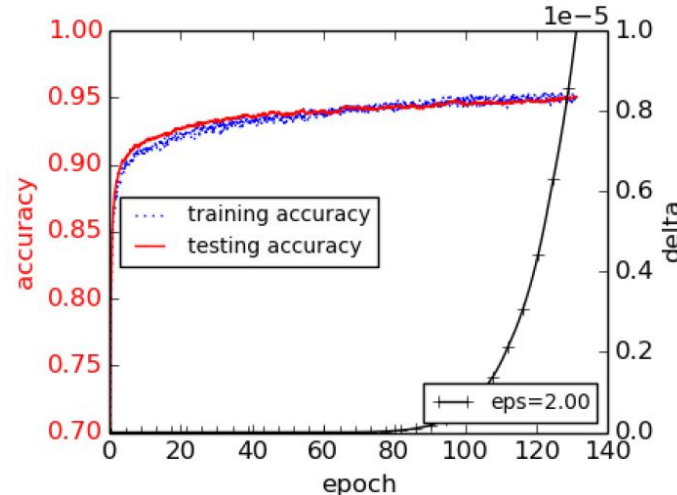
11.2.3.1 Differentially Private SGD

- MNIST dataset
 - Training and testing accuracies for three different levels of Gaussian noise with variances $\sigma \in \{8, 4, 2\}$, and clipping threshold $C = 4$
 - The corresponding privacy budget values are set to $\epsilon \in \{0.5, 2, 8\}$
 - The upper bound for the privacy probability parameter is set to $\delta = 10^{-5}$
 - Thus, the (ϵ, δ) -differential privacies are $(0.5, 10^{-5})$, $(2, 10^{-5})$, $(8, 10^{-5})$
 - Moment Accountant allows to obtain a δ value for a fixed privacy budget ϵ
 - The obtained test set accuracies are 90%, 95%, and 97%, respectively
 - Larger noise achieves lower test accuracy, but provides increased privacy protection

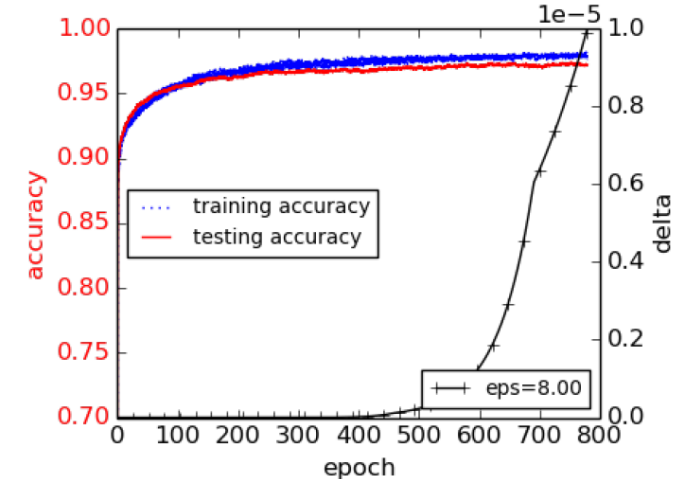
Large noise, $\epsilon = 0.5$



Medium noise, $\epsilon = 2$



Small noise, $\epsilon = 8$





Privacy versus Accuracy Trade-Off

11.2.3.1 Differentially Private SGD

- Perfect privacy in ML models is not possible
 - Adding too much noise to the model parameters would diminish the accuracy and the usefulness of the model
 - There is a trade-off between privacy protection and accuracy
- DP-SGD achieves privacy protection for deep NNs with a small decrease in the model accuracy and small increase in the training complexity
 - The approach adds Gaussian noise to the gradients in SGD to reduce the possibility for memorization of individual input instances by the model

Distributed Learning

11.2.4 Distributed Learning

- **Distributed learning** allows multiple parties to train a global model without releasing their private data
- Some form of **aggregation** is applied to the local updates of the model parameters by the users in distributed learning to create a global model
 - E.g., averaging is one common form of aggregation
- **Federated learning** is the most popular distributed learning scheme

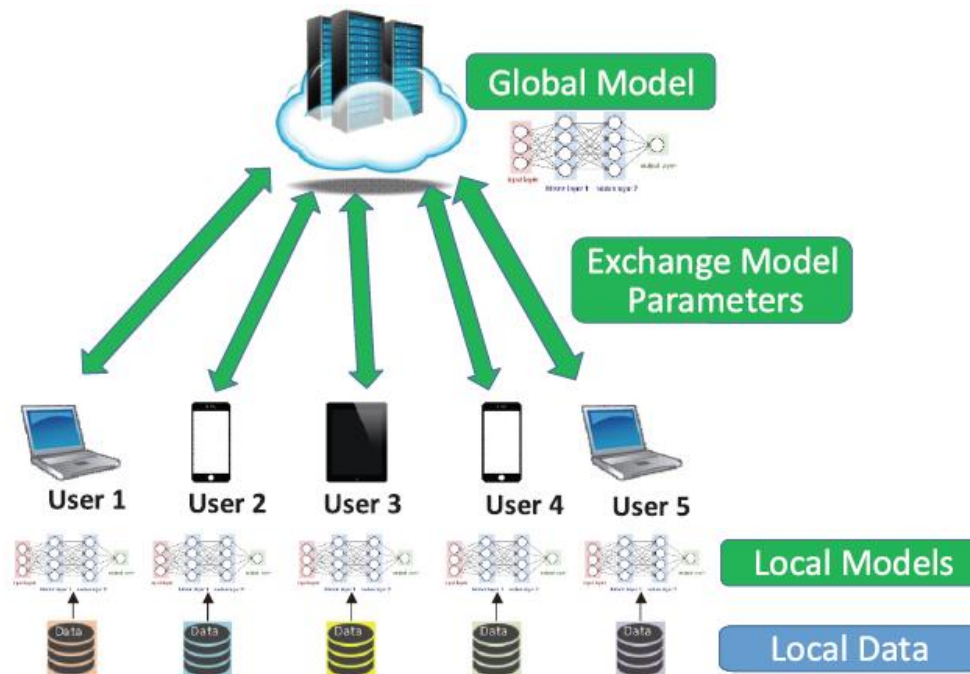


Figure from: Liu et al. (2020) When Machine Learning Meets Privacy: A Survey and Outlook

Distributed Learning

11.2.4 Distributed Learning

- *Federated learning* or *collaborative learning* – learn one global model using data stored at multiple locations (e.g., remote devices)
 - The data are processed locally, and used to update the model
 - The data do not leave the remote devices, remains private
 - The central server aggregates the updates and creates the global model
- *Decentralized Peer-to-Peer (P2P) learning* – the remote devices communicate and exchange the updates directly, **without a central server**
 - Removes the need to send updates to a potentially untrusted central server
- *Split learning* – each remote device is used to **train several layers** of the global model, and send the outputs to a central server
 - The remote devices can train the initial layers of a DNN, and the central server can train the final layers
 - The gradient is back-propagated from the central server to each user to sequentially complete the back-propagation through all layers of the model
 - The devices send the intermediate layers outputs, rather than model parameters
 - Split learning is more common for IoT devices with limited computational resources

ML-Specific Techniques

11.2.5 ML-specific Techniques

- In the lecture on privacy attacks in ML, we mentioned that overfitting is one of the reasons for information leakage
- *Regularization techniques* in ML can therefore be used to reduce overfitting, as well as a defense strategy
 - Different regularization techniques in NNs include:
 - **Explicit regularization**: dropout, early stopping, weight decay
 - **Implicit regularization**: batch normalization
- Other ML-specific techniques include:
 - Dimensionality reduction – removing inputs with features that occur rarely in the training set
 - Weight-normalization – rescaling the weights of the model during training
 - Selective gradient sharing – in federated learning, the users share a fraction of the gradient at each update

P A P E R R E V I E W · I C M L 2 0 2 5

Scaling Laws for *Differentially Private* Language Models

McKenna, Huang, Sinha, Balle, Charles, Choquette-Choo,
Ghazi, Kaissis, Kumar, Liu, Yu, Zhang · Google Research & DeepMind

Adversarial ML · Presented by Tom

THE CENTRAL TENSION

We need user data. We must protect it.



Why user data matters

- Reflects real-world use better than web scrape
- Critical for RLHF and synthetic data generation
- Modern frontier models depend on it



Why privacy matters

- LLMs memorize and regurgitate training data
- Membership inference & extraction attacks
- DP is the principled, provable mitigation

→ *DP resolves the tension in principle, but breaks standard LLM training dynamics.*

QUICK REFRESHER

Differential Privacy via DP-SGD

(ϵ, δ) -DP: adjacent datasets D, D' produce nearly indistinguishable outputs.

ϵ = privacy budget · δ = failure probability · smaller = more private

01



Clip

Per-example gradients bounded to L2 norm C . Limits any one record's influence.

02



Noise

Gaussian noise $N(0, \sigma^2 C^2 I)$ added to the batch gradient sum.

03

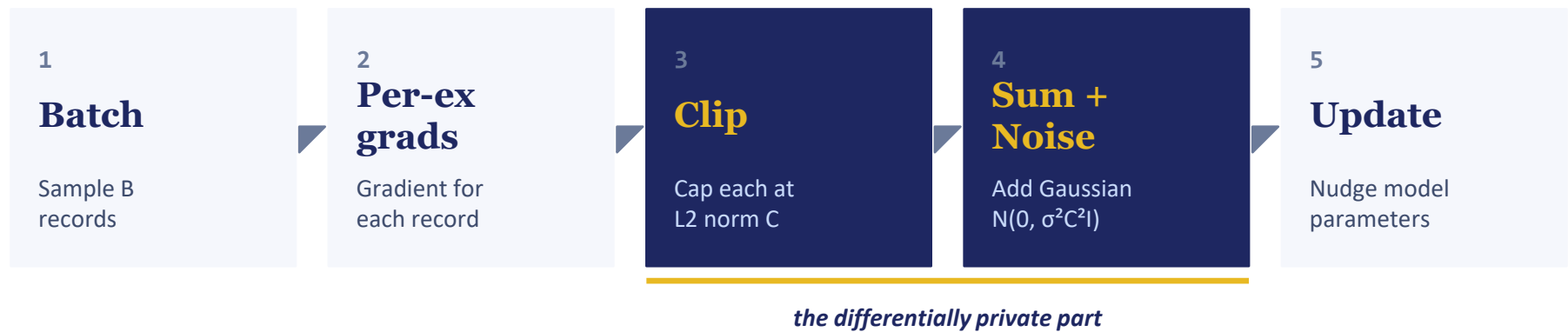


Account

Compose privacy loss across steps → total (ϵ, δ) budget.

THE PIPELINE

One DP-SGD step, from batch to update



Accounting runs in the background

Every step spends a small amount of ϵ . After T iterations, a privacy accountant composes all the small losses into one total (ϵ, δ) budget for the whole run. You pick the budget up front; the accountant tells you when you've used it.

THE GAP

Chinchilla doesn't survive DP.

Non-private

Two budgets

- Model size (N)
- Training tokens (D)

Chinchilla: scale N and D together, ~20 tokens per parameter.

Under DP-SGD

Three budgets

- Model size (N)
- Data budget (tokens)
- Privacy budget (ϵ , δ)

Noise destabilizes training; batch size becomes a first-class lever.

So — how do we pick N, batch size, iterations, and ϵ jointly?

WHERE THIS FITS

Standing on several sets of shoulders

2006

Differential Privacy

Dwork et al. formalize ϵ -DP — the mathematical guarantee everything downstream rests on.

2016

DP-SGD

Abadi et al. bring DP to deep learning via per-example clipping + Gaussian noise + moments accountant.

2022

Chinchilla

Hoffmann et al. establish non-private LLM scaling laws: ~20 tokens per parameter is compute-optimal.

2023

TAN: Scaling Laws of DP-SGD

Sander et al. first to study DP-SGD scaling, but limited to image models and narrower regimes.

2025 • **This paper** — unifies DP training dynamics with modern LLM scaling, validated at 1B params.

THE KEY INSIGHT

One ratio captures (almost) everything.

noise - batch ratio

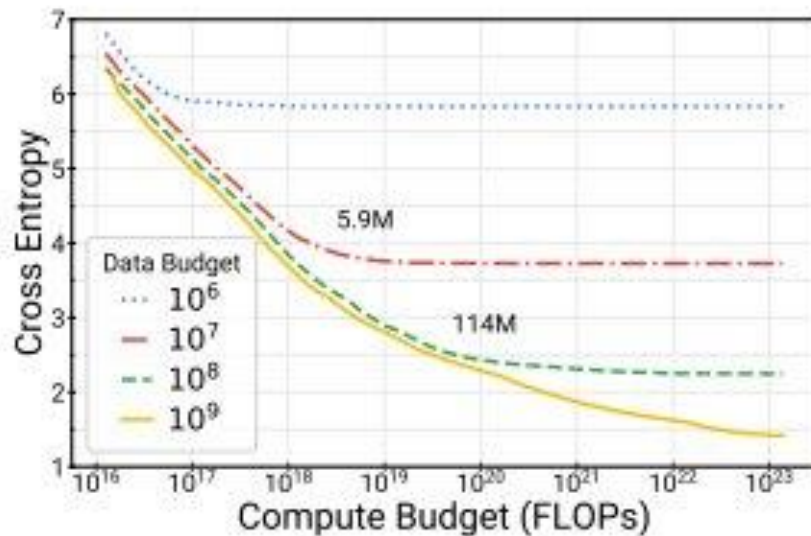
$$\sigma_{\text{effective}} = \sigma_{\text{privacy}} / B$$

The claim

Loss depends primarily on **model size N**, **iterations T**, and the **noise-batch ratio**. Compute, privacy, and data budgets influence loss *only through their effect on this ratio*.

Why it works: DP Gaussian noise dominates the natural stochasticity of mini-batch sampling, so you can ignore the sampling noise and treat privacy noise as the only source of gradient noise.

The critical compute budget



(a) Privacy Budget: $\epsilon = 1$

How to read it

- Each curve = a different data budget N .
- Loss drops as compute grows — until it doesn't.
- The flatline is the **critical compute budget** — adding more compute helps nothing unless you also raise ϵ or N .

Unlike non-private scaling laws, throwing more compute at DP training saturates fast.

M E T H O D O L O G Y

How they fit the scaling law



Experimental Setup

Architecture: Decoder-only transformer (BERT-style family)

Model sizes: ~4M (Tiny) to ~778M (Mega) params

Corpus: ~3.3B words

Optimizer: Generalized DP-SGD (clip + Gaussian)

Sweeps: $N \times \text{iterations} \times \text{batch size} \times \sigma$

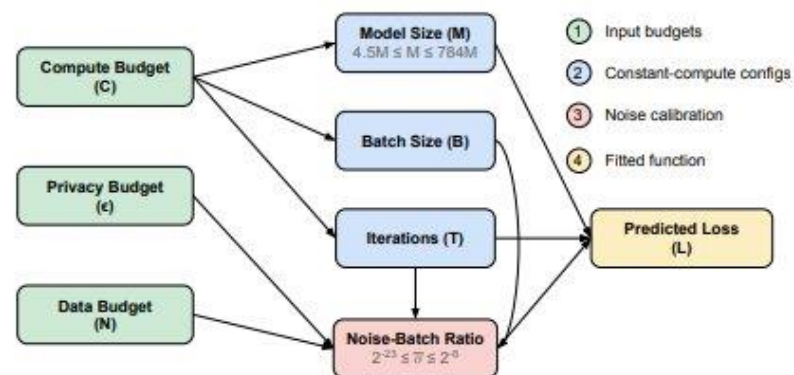


Figure 2: compute, privacy, and data budgets → noise-batch ratio → predicted loss.

KEY FINDINGS

Four results that reshape DP training

01



Batch size is the lever

Under DP, large batches dominate — they drive $\sigma_{\text{effective}}$ down. Unlike non-private training, bigger batches materially improve the privacy-utility tradeoff.

02



ϵ alone has diminishing returns

Increasing the privacy budget in isolation helps little. You must couple it with more compute or more data.

03



Compute-optimal shifts

DP prescribes smaller models trained longer on larger batches — departing from the Chinchilla recipe for a given compute budget.

04

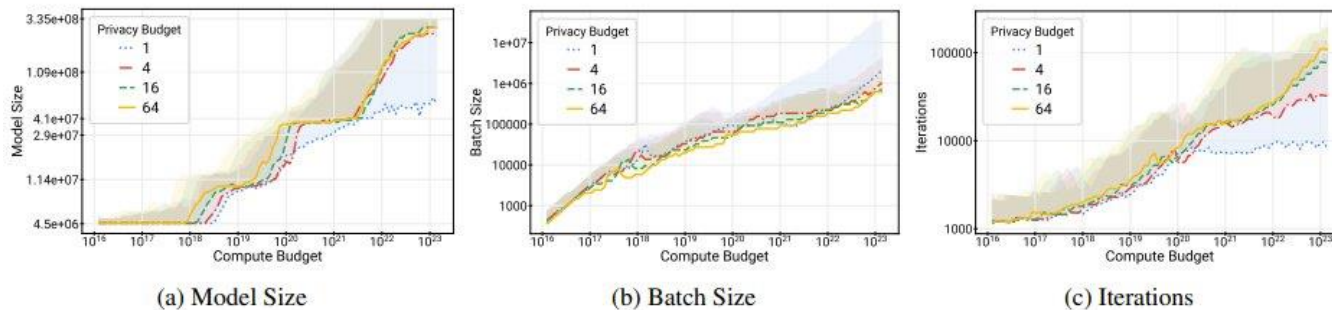


Stability is a scaling axis

DP noise induces loss spikes and divergence at scale. The laws explicitly account for training stability, not just final loss.

THE RECIPE, VISUALIZED

Optimal N, B, and T for your privacy and compute budget



How to read it

→ Three panels: model size N, batch size B, iterations T.

→ Lines = optimal value at each (ϵ , compute) pair.

→ Shaded = within 1% of optimal loss.

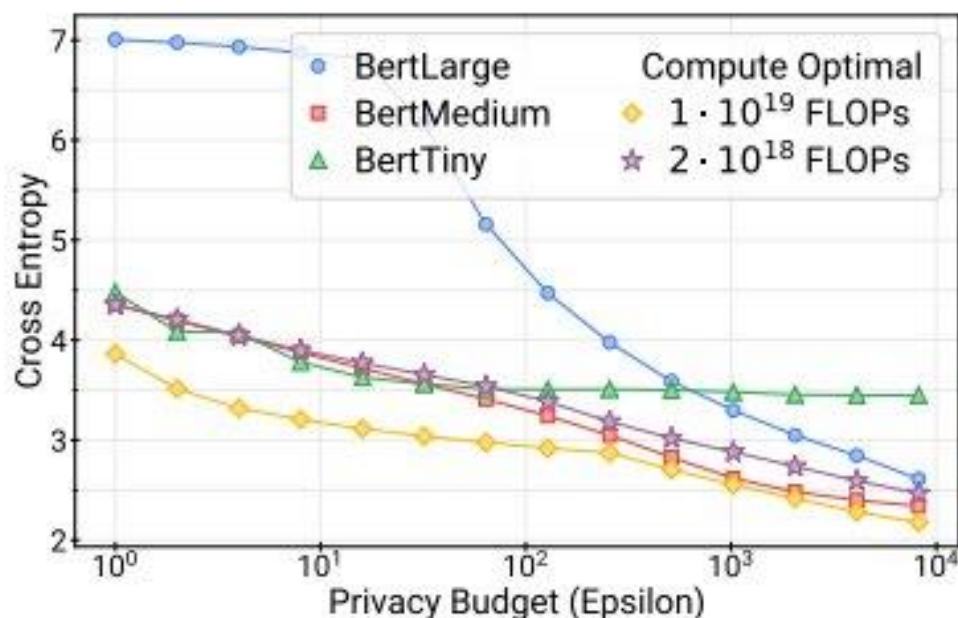
The punchline

As ϵ tightens, optimal **batch size grows** and optimal **model size shrinks** — the three findings on the previous slide, shown in one picture.

Figure 1, McKenna et al. 2025. Fixed data budget of 10^8 tokens.

DOES IT BEAT THE BASELINES?

Compute-optimal vs. standard baselines at 10^{19} FLOPs



Three baselines tested

- BertLarge — Chinchilla-style config
- BertMedium — larger batches
- BertTiny — DP-expert config
- *Compute-optimal (scaling-law)*

5×

compute savings at matched utility

vs. the best baseline, across reasonable privacy budgets

Naively fixing a training config across all privacy budgets — the norm in prior DP literature — leaves huge utility on the table.

Why this matters for threat modeling

The threat

Membership inference · training data extraction · model inversion — all exploit the fact that LLMs memorize individual records.



Provable defense

DP provides a worst-case guarantee against any adversary — not an empirical hope.



Predictable cost

These scaling laws let you price privacy in advance: compute, data, and ϵ needed for target utility.



Design target

Defenders can now pick operating points with known tradeoffs instead of training blind.

DP is no longer a research curiosity — these laws make it an engineering decision.

VALIDATION IN THE WILD

VaultGemma — the laws worked.

1B

parameters, trained from scratch with DP

$\epsilon \leq 2.0$ · $\delta \leq 1.1e-10$

sequence-level DP guarantee over 1024-token sequences

Outcomes

Predicted vs actual loss: final training loss landed remarkably close to what the scaling laws forecast.

Open release: weights on Hugging Face and Kaggle — reproducible baseline for the community.

Built on Gemma 2 training mixture, showing the laws transfer to production-grade data.

research.google/blog/vaultgemma

H O N E S T C A V E A T S

Where this framework stops working

Sequence-level, not user-level



VaultGemma's guarantee protects 1024-token sequences, not whole users. If one user contributes many sequences, their overall exposure exceeds ϵ .

Pretraining only



The laws fit decoder-only pretraining from scratch. Fine-tuning, instruction tuning, and RLHF have different dynamics and aren't covered.

Ratio assumption may break



The "only the noise-batch ratio matters" simplification assumes privacy noise dominates sampling noise. At very large B or very loose ϵ , this can fail.

Downstream utility unmeasured



The laws predict training loss. Whether that loss translates to good task accuracy, reasoning, or alignment is an open empirical question.

T A K E A W A Y S

What to remember

1

One ratio, not three budgets.

Privacy, compute, and data interact through the noise-batch ratio — that's the single number to reason about.

2

DP rewrites compute-optimal choices.

Smaller models, longer training, much larger batches. Don't port Chinchilla recipes blindly.

3

Provable privacy is now practical.

VaultGemma demonstrates a 1B-param model trained from scratch at $\epsilon \leq 2$ — with predictable utility.

Discussion: where does this framework break?

References

1. Liu et al. (2020) When Machine Learning Meets Privacy: A Survey and Outlook ([link](#))
2. Rigaki and Carcia (2021) A Survey of Privacy Attacks in Machine Learning ([link](#))
3. Cristofaro (2020) An Overview of Privacy in Machine Learning ([link](#))
4. Zhang (2025) On Large Language Models Safety, Security, and Privacy: A Survey ([link](#))
5. Liu et al. (2020) When Machine Learning Meets Privacy: A Survey and Outlook ([link](#))
6. Rigaki and Carcia (2021) A Survey of Privacy Attacks in Machine Learning ([link](#))
7. Cristofaro (2020) An Overview of Privacy in Machine Learning ([link](#))
8. Borealis AI Tutorial #13: Differential Privacy II: Machine Learning and Data Generation ([link](#))
9. Davide Testuggine and Ilya Mironov blog: Differential Privacy Series Part 1-DP-SGD Algorithm Explained ([link](#))
10. Joseph P. Near and Chiké Abuah, Programming Differential Privacy – Chapter III: Differential Privacy ([link](#))